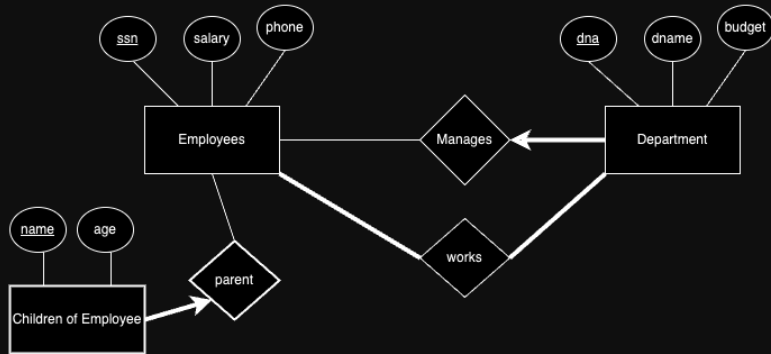
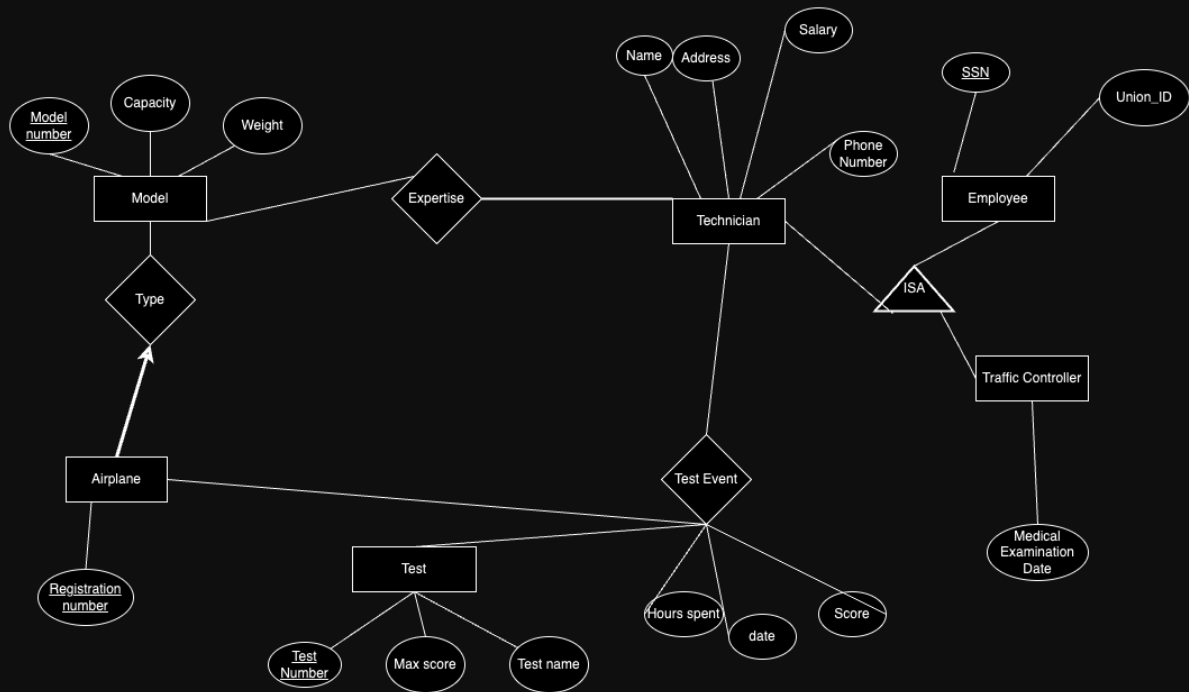


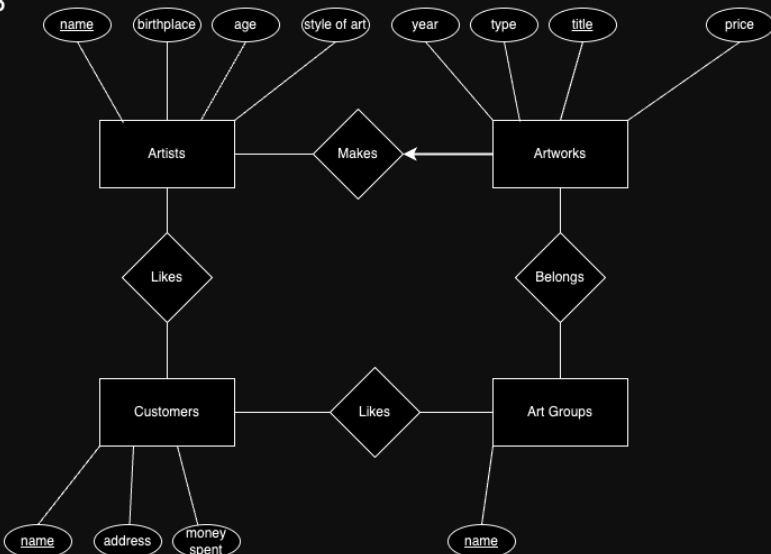
2.4



2.6



2.8



3.14

```
CREATE TABLE Employees(  
SSN CHAR(9),  
Salary INTEGER,  
phone CHAR(10),  
PRIMARY KEY (SSN) );
```

```
CREATE TABLE Departments(  
DNA INTEGER,  
Dbudget INTEGER,  
Dname VARCHAR(20),  
PRIMARY KEY (DNA) );
```

```
CREATE TABLE Works_in(  
SSN CHAR(9),  
DNA INTEGER,  
PRIMARY KEY (SSN, DNA),  
FOREIGN KEY (SSN) REFERENCES Employees,  
FOREIGN KEY (DNA) REFERENCES Departments);
```

```
CREATE TABLE Manages(  
SSN CHAR(10),  
DNA INTEGER,  
PRIMARY KEY (DNA),  
FOREIGN KEY (SSN) REFERENCES Employees,  
FOREIGN KEY (DNA) REFERENCES Departments);
```

```
CREATE TABLE Child_of(SSN CHAR(10),  
Name VARCHAR(10),  
Age INTEGER,  
PRIMARY KEY (SSN, Name),  
FOREIGN KEY (SSN)  
REFERENCES Employees  
ON DELETE CASCADE )
```

Participation Constraints (Total Participation) like the relationship between Employees and Departments, SQL is hard to guarantee every employee must have a department.

3.16

```
CREATE TABLE Airplane (  
registration_no INT PRIMARY KEY,  
model_no INT NOT NULL,
```

```
FOREIGN KEY (model_no) REFERENCES Models(model_no)
);
```

```
CREATE TABLE Model (
model_no INT PRIMARY KEY,
capacity INT NOT NULL ,
weight DECIMAL(10,2) NOT NULL
);
```

```
CREATE TABLE Employee (
ssn VARCHAR(20) PRIMARY KEY ,
union_id VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE Technician (
employee_ssn VARCHAR(20) PRIMARY KEY,
name VARCHAR(50) NOT NULL ,
address VARCHAR(100) NOT NULL ,
phone_no VARCHAR(20) NOT NULL ,
salary DECIMAL(10, 2) NOT NULL ,
FOREIGN KEY (employee_ssn) REFERENCES Employees(ssn)
);
```

```
CREATE TABLE TrafficController (
employee_ssn VARCHAR(20) PRIMARY KEY,
most_recent_exam DATE NOT NULL ,
FOREIGN KEY (employee_ssn) REFERENCES Employees(ssn)
);
```

```
CREATE TABLE Expertise(
employee_ssn VARCHAR(20),
model_no INT,
FOREIGN KEY (employee_ssn) REFERENCES Employees(ssn),
FOREIGN KEY (model_no) REFERENCES Models(model_no),
PRIMARY KEY (employee_ssn, model_no)
);
```

```
CREATE TABLE Test (
test_no INT PRIMARY KEY,
name VARCHAR(50) NOT NULL ,
max_score INT
);
```

```
CREATE TABLE Test_Event (
```

```

employee_ssn VARCHAR(20) NOT NULL ,
test_no INT NOT NULL ,
registration_no INT NOT NULL ,
score INT NOT NULL ,
hours_spent INT NOT NULL ,
date DATE NOT NULL ,
FOREIGN KEY (employee_ssn) REFERENCES Employees(ssn),
FOREIGN KEY (test_no) REFERENCES Tests(test_no),
FOREIGN KEY (registration_no) REFERENCES Planes(registration_no)
);

```

4.4

- 1) Select the names of all suppliers who supply red parts that cost less than 100
- 3) Select the names of the suppliers, where suppliers under the same name must satisfy that at least one of them has a red part cost less than 100 and at least one of them has green part cost less than 100.
- 5) Select the names of all suppliers who supply both red parts and green part and both red and green cost less than 100

5.4

- 2)


```

SELECT W.did, COUNT(W.eid) AS num_employees
FROM Works W
GROUP BY W.did
HAVING SUM(W.pct_time) > 20;

```
- 3)


```

SELECT E.ename
FROM Emp E
WHERE E.salary > ALL (
  SELECT D.budget
  FROM Works W, Dept D
  WHERE E.eid = W.eid AND W.did = D.did
);

```
- 5).


```

SELECT E.ename
FROM Emp E, Dept D
WHERE E.eid = D.managerid
AND D.budget = (SELECT MAX(budget) FROM Dept);

```
6.


```

SELECT D.managerid
FROM Dept D
GROUP BY D.managerid
HAVING SUM(D.budget) > 5000000;

```

5.8

1.

```
CREATE TABLE Students(  
snum INTEGER,  
sname CHAR(20),  
major CHAR(20),  
level CHAR(20),  
age INTEGER,  
PRIMARY KEY (snum));
```

```
CREATE TABLE Faculty (  
fid INTEGER,  
fname CHAR(20),  
deptid INTEGER,  
PRIMARY KEY (fid));
```

```
CREATE TABLE Class (  
name CHAR(20),  
meets_at TIME,  
room CHAR(20),  
fid INTEGER,  
PRIMARY KEY(name),  
FOREIGN KEY (fid) REFERENCES Faculty );
```

```
CREATE TABLE Enrollment (  
snum INTEGER,  
cname CHAR(20),  
PRIMARY KEY (snum,cname),  
FOREIGN KEY (snum) REFERENCES Student ,  
FOREIGN KEY (cname) REFERENCES Class );
```

2.

(a)

```
CREATE TABLE Enrollment (  
snum INTEGER,  
cname CHAR(20),  
PRIMARY KEY (snum,cname),  
FOREIGN KEY (snum) REFERENCES Student ,  
FOREIGN KEY (cname) REFERENCES Class ,  
CHECK (( SELECT COUNT(E.snum)  
FROM Enrollment E  
GROUP BY E.cname)>5),  
CHECK (( SELECT COUNT(E.snum)
```

```
FROM Enrollment E
GROUP BY E.cname)<=30) );
```

(c)

```
CREATE ASSERTION TeachTwo
CHECK((SELECT COUNT(*)
      FROM Faculty F, Class C
      WHERE F.fid = C.fid
      GROUP BY C.fid
      HAVING COUNT(*)<2) = 0);
```

(e)

```
CREATE ASSERTION InMath101
CHECK(( SELECT COUNT(*)
      FROM Student S
      WHERE S.snum NOT IN ( SELECT E.snum
                           FROM Enrollment E
                           WHERE E.cname = 'Math101'))=0);
```

(g)

```
CREATE TABLE Class (
      name CHAR(20),
      meets_at TIME,
      room CHAR(20),
      fid INTEGER,
      PRIMARY KEY(name),
      FOREIGN KEY (fid) REFERENCES Faculty,
      CHECK(( SELECT C.room,C.meets
            FROM Class C
            GROUP BY C.room,C.meets
            HAVING COUNT(*)>1))=0);
```

(l)

```
CREATE ASSERTION MoreCSMajor
CHECK( (SELECT COUNT(E.cname)
      FROM Enrollemtn E,Students S
      WHERE S.snum=E.snum AND S.major>'CS')>
      (SELECT COUNT(E.cname)
      FROM Enrollemtn E,Students S
      WHERE S.snum=E.snum AND S.major>'Math'));
```

(m)

```
CREATE ASSERTION MoreEnrolledMath
CHECK(( SELECT COUNT(E.snum)
```

```
FROM Enrollment E, Faculty F, Class C
WHERE E.cname=C.name AND
C.fid=F.fid AND F.deptid=33)>
(SELECT COUNT(E.snum)
FROM Student S
WHERE S.Major='Math'));
```

8.6

Refer to page 291 of textbook

8.10

- 1) We could use the clustered index on eid as key and make it unique.
- 2) If we want to change the did of employees whose eid was in a range, we would be able to quickly locate eid's surroundings.
- 3) updating unclustered index would cost more.
For instance, if you need to delete all employees older than 60 from the Emp table, the system would first use the unclustered index on age to identify which rows need to be deleted.
- 4) An update that modifies a field not indexed will not be influenced.
For instance, changing the department (did) of an employee, would neither benefit from the available indexes nor be slowed down by them.